



WOLLO University

Department of software engineering

Blueprint based exit material for

Operating system and system programming

Prepared by:

@Neyatibeb

KOMBOLCHA, Ethiopia

2015 E.C

❖ Function of modern operating system

- The functions of a modern operating system include:
 - Resource management: The operating system manages the resources of the computer, such as the CPU, memory, and storage. This includes allocating resources to processes, ensuring that each process has access to the resources it needs, and preventing processes from interfering with each other.
 - Process management: The operating system manages the processes that are running on the computer. This includes creating and destroying processes, scheduling processes for execution, and providing mechanisms for communication and synchronization between processes.
 - Memory management: The operating system manages the memory of the computer, ensuring that each process has access to the memory it needs and that memory is used efficiently. This includes allocating and deallocating memory to processes, and managing the paging of memory between main memory and disk storage.
 - File management: The operating system manages the files on the computer, ensuring that they are stored and accessed efficiently. This includes creating and deleting files, reading and writing files, and managing file permissions.
 - Device management: The operating system manages the devices that are connected to the computer, such as the keyboard, mouse, and printer. This includes providing drivers for devices, and managing the input and output of data to and from devices.
 - Security: The operating system provides security for the computer, protecting it from unauthorized access and malicious software. This includes enforcing access control, detecting and preventing malware, and backing up data.
 - User interface: The operating system provides a user interface that allows users to interact with the computer. This includes graphical user interfaces (GUIs) and command-line interfaces (CLIs).
 - Networking: The operating system provides networking capabilities, allowing computers to communicate with each other over a network.
 - Multitasking: The operating system allows multiple processes to run simultaneously, sharing the resources of the computer.
 - Scheduling: The operating system schedules the execution of processes, ensuring that each process gets a fair share of the CPU time.
 - Error handling: The operating system handles errors that occur during the execution of processes, preventing the computer from crashing.

These are just some of the most important functions of modern operating systems. There are many other functions that are performed by operating systems, but these are some of the most essential ones.

Modern operating systems are complex pieces of software that perform a wide variety of tasks. The functions that I have listed above are essential for the smooth operation of any computer system.

❖ Condition that lead to deadlock

Deadlock is a set of processes is deadlocked if each process in the set is waiting for an event that only another process in the set can cause. Deadlocks are a set of blocked processes each holding a resource and waiting to acquire a resource held by another process.

- There are four necessary conditions for deadlock to occur in an operating system:
 - Mutual exclusion:

Department of software engineering

- Each resource is either currently assigned to exactly one process or is available
- Each resource must be held exclusively by one process at a time.
- Hold and wait:
 - Process currently holding resources granted earlier can request more resources.
 - A process must be holding at least one resource and waiting to acquire additional resources that are held by other processes.
- No pre-emption:
 - Previously granted resources cannot be forcibly taken away from process.
 - A resource cannot be taken away from a process that is currently holding it
- Circular wait:
 - There must be a circular chain of processes, each of which is waiting for a resource held by the next process in the chain.

If all four of these conditions are met, then a deadlock will occur. The processes in the circular chain will be blocked forever, waiting for each other to release resources.

There are a number of ways to prevent deadlocks, such as:

- Avoiding mutual exclusion: This can be done by using a shared-memory model, where all processes have access to all resources.
- Breaking the hold and wait condition: This can be done by forcing processes to release resources as soon as they are no longer needed.
- Allowing pre - emption: This means that resources can be taken away from processes that are not using them.
- Breaking the circular wait condition: This can be done by rearranging the order in which processes request resources.
- Deadlocks can also be detected and recovered from. Once a deadlock has been detected, the operating system can take steps to break the circular chain of processes and allow them to continue execution.

Here are some examples of deadlocks:

- Two processes are each holding a file that the other process needs.
- Two processes are each waiting to use the same printer.
- Two processes are each waiting for the other process to finish using the CPU.

Deadlocks can be a serious problem in operating systems, as they can prevent any progress from being made. However, there are a number of ways to prevent and recover from deadlocks.

❖ Process and scheduling algorithm

Process scheduling is the activity of the process manager that handles suspension of running process from CPU and selection of another process on the basis of a particular strategy. The part of operating system that makes the choice is called scheduler. The algorithm used by this scheduler is called scheduling algorithm. Process scheduling is an essential part of a multiprogramming operating systems.

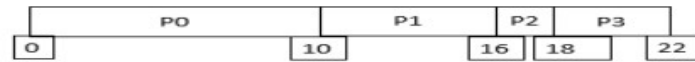
Objectives (goals) of scheduling

- Fairness: giving each process a fair share of the CPU.

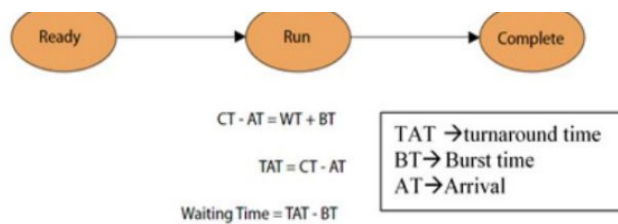
▪ Example

Process	Arrival Time (T ₀)	Burst Time (ΔT)	Finish Time (T ₁)	Turnaround Time (TAT = T ₁ -T ₀)	Waiting Time (WT = TAT-ΔT)
P0	0	10	10	10	0
P1	1	6	16	15	9
P2	3	2	18	15	13
P3	5	4	22	17	13

▪ Gantt Chart



- Average Turnaround Time: $(10+15+15+17)/4 = 14.25$ ms.
- Average Waiting Time: $(0+9+13+13)/4 = 8.75$ ms.



- Scheduling algorithms Modes
 - Preemptive algorithm
 - If a process is still running at the end of its time interval, it is suspended and another process is picked up
 - Non-preemptive
 - Picks a process to run and then lets it run till it blocks or it voluntarily releases the CPU
 - once the CPU has been allocated to a process, the process keeps CPU until it releases the CPU either by terminating or by switching to the waiting state.
- Some of the most common scheduling algorithms used in operating systems:
 1. First Come First Served (FCFS)
 - The process that request first is served first. It means that processes are served in the exact order of their arrival. Decision Mode is Non preemptive: Once a process is selected, it runs until it is blocked for an I/O or some other event or it is terminated. This strategy can be easily implemented by using FIFO (First In First Out) queue. When CPU becomes free, a process from the first position in a queue is selected to run. Disadvantages of this algorithm are that Convoy effect is possible. All small I/O bound processes wait for one big CPU bound process to acquire CPU.
 - This is the simplest scheduling algorithm. Processes are executed in the order in which they arrive.

2. Shortest
- The pro
Mode is
I/O or s
FIFO (F
on their
queue is
Starvati

Process	Arrival Time (T ₀)	Burst Time (ΔT)	Finish Time (T ₁)	Turnaround Time (TAT = T ₁ -T ₀)	Waiting Time (WT = TAT-ΔT)
P0	0	10	22	22	12
P1	1	6	9	8	2
P2	3	2	5	2	0
P3	5	4	13	8	4

- Gantt Chart

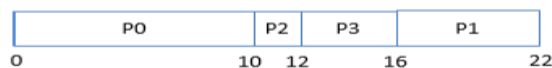


- Average Turnaround Time: $(22+8+2+8) / 4 = 10$ ms.
- Average Waiting Time: $(12+2+0+4)/4 = 4.5$ ms.

3. S
- T
v
a
p
tl
p
b
p
i:

Process	Arrival Time (T ₀)	Burst Time (ΔT)	Finish Time (T ₁)	Turnaround Time (TAT = T ₁ -T ₀)	Waiting Time (WT = TAT-ΔT)
P0	0	10	10	10	0
P1	1	6	22	21	15
P2	3	2	12	9	7
P3	5	4	16	11	7

- Gantt Chart



- Average Turnaround Time: $(10+21+9+11)/4 = 12.75$ ms.
- Average Waiting Time: $(0+15+7+7)/4 = 7.25$ ms.

ve
cess
ew
ended and
ueue. All
n CPU
rrvation is
overhead

4. Round Robin (RR)

Each selected process is assigned a time interval, called time quantum or time slice. Process is allowed to run only for this time interval. Here, two things are possible. First, process is either blocked or terminated before the quantum has elapsed. In this case the CPU switching is done and another process is selected to run.

In this case, the process is moved to the back of the queue. If its time quantum is over, it is over the queue. If it is over the queue, it is over the queue.

Process	Arrival Time (T ₀)	Burst Time (ΔT)	Finish Time (T ₁)	Turnaround Time (TAT = T ₁ -T ₀)	Waiting Time (WT = TAT-ΔT)
P0	0	10	10	10	0
P1	1	6	22	21	15
P2	3	2	16	13	11
P3	5	4	14	9	5

- Gantt Chart (small values for priority means higher priority of a process)

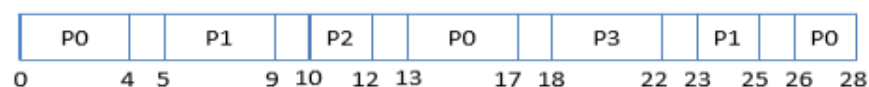


- Average Turnaround Time: $(10+21+13+9) / 4 = 13.25 \text{ ms}$
- Average Waiting Time: $(0+15+11+5) / 4 = 7.75 \text{ ms}$

If the time quantum is too short, it causes frequent context switches and lowers CPU efficiency. If it is too long, it causes poor response for short interactive process.

Process	Arrival Time (T ₀)	Burst Time (ΔT)	Finish Time (T ₁)	Turnaround Time (TAT = T ₁ -T ₀)	Waiting Time (WT = TAT-ΔT)
P0	0	10	28	28	18
P1	1	6	25	24	18
P2	3	2	12	9	7
P3	5	4	22	17	13

- Gantt Chart (Quantum time is 4 ms & context switch overhead is 1 ms)



- Average Turnaround Time: $(28+24+9+17)/4 = 19.5 \text{ ms}$.
- Average Waiting Time: $(18+18+7+13)/4 = 14 \text{ ms}$.

The process, that has highest priority, is served first. The Decision Mode: Non Preemptive: Once a process is selected, it runs until it blocks for an I/O or some event or it terminates. This strategy can be implemented by using sorted FIFO queue. All processes in a queue are sorted based on their priority with highest priority process at front end. When CPU becomes free, a process from the first position in a queue is selected to run. Here Starvation is possible for low priority processes. It can be overcome by using technique called 'Aging'. Aging: gradually increases the priority of processes that wait in the system for a long time.

• Preemptive Priority Scheduling

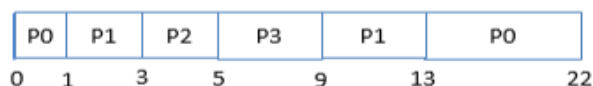
The process, that has highest priority, is served first. The Decision Mode is Preemptive: When a new process arrives, its priority is compared with current process priority. If the new process has higher priority than the current, the current process is suspended and new job is started. This strategy can be implemented by using sorted FIFO queue. All processes in a queue are sorted based on priority with highest priority process at front end. When CPU becomes free, a process from the first position in a queue is selected to run. Here again Starvation is possible for low priority processes. It can be overcome by using technique called 'Aging'. Aging: gradually increases the priority of processes that wait in the system for a long time. Context switch overhead is there.

Each of these scheduling algorithms has its own advantages and disadvantages. FCFS is simple to implement but it can lead to starvation, where long running processes never get a chance to run. SJF is optimal for minimizing the average waiting time of a process but it can lead to starvation of a low priority process. Round Robin is a simple to implement and fair scheduling algorithm but it can lead to starvation of a low priority process. Priority Scheduling is a simple to implement and fair scheduling algorithm but it can lead to starvation of a low priority process.

Process	Arrival Time (T ₀)	Burst Time (ΔT)	Finish Time (T ₁)	Turnaround Time (TAT = T ₁ -T ₀)	Waiting Time (WT = TAT-ΔT)
P0	0	10	22	22	12
P1	1	6	13	12	6
P2	3	2	5	2	0
P3	5	4	9	4	0

The system while a

- Gantt Chart (small values for priority means higher priority of a process)



Here at

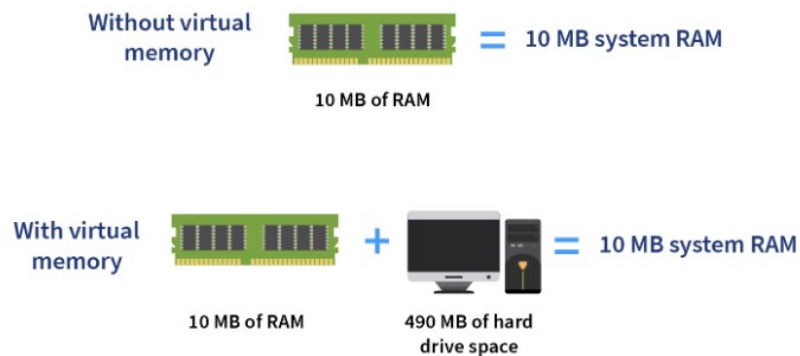
- Average Turnaround Time: $(22+12+2+4) / 4 = 10 \text{ ms}$
- Average Waiting Time: $(12+6+0+0) / 4 = 4.5 \text{ ms}$
- Throughput: The number of processes that can be completed per unit time.
- Fairness: The extent to which all processes are given a fair chance to run.
- Complexity: The ease with which the algorithm can be implemented and maintained.

Department of software engineering

The choice of scheduling algorithm is a trade-off between these factors. Some algorithms, such as SJF, can achieve optimal turnaround time, but they can be unfair to long-running processes. Other algorithms, such as RR, are fair, but they can lead to lower throughput. The best algorithm for a particular system will depend on the specific requirements of that system.

❖ Concept of virtual memory

How Virtual Memory Works?



Assume that an operating system uses 500 MB of RAM to hold all of the running processes. However, there is now only 10 MB of actual capacity accessible on the RAM. The operating system will then allocate 490 MB of virtual memory and manage it with an application called the Virtual Memory Manager (VMM). As a result, the VMM will generate a 490 MB file on the hard disc to contain the extra RAM that is necessary. The OS will now proceed to address memory, even if only 10 MB of space is available because it considers 500 MB of actual memory saved in RAM. It is the VMM's responsibility to handle 500 MB of memory, even if only 10 MB is available.

Advantage of virtual memory

- Increases the degree of multiprogramming: You can run many applications at once without buying more memory RAMs.
- **Data Sharing:** Data that is common can be shared between memory.
- **Avoids Relocation:** The code in the physical memory can be accessed whenever required without relocation of the code.
- **Increases memory:** Users can run larger applications in systems that cannot support further RAM. Thus, it can run applications that are larger than the physical memory.
- **Increases effective use of CPU:** Since more processes can be maintained in the memory, the CPU is more effectively used. Each page is stored on the disk while it's in use, and further removed.

❖ Basic component of OS and its purpose

- There are 8 components of an operating system those are
 - Process Management
 - I/O Device Management
 - File Management
 - Network Management
 - Main Memory Management
 - Secondary Storage Management
 - Security Management

Department of software engineering

- Command Interpreter System
- Process management is responsible for creating and managing processes, which are programs that are currently running on the computer. The operating system ensures that processes are executed in a safe and efficient manner, and that they do not interfere with each other.
- File management is responsible for storing and organizing files on the computer's disk drives. The operating system provides a way for users to create, delete, and access files, and it also ensures that files are not corrupted.
- I/O device management is responsible for managing the computer's input and output devices, such as keyboards, mice, printers, and hard drives. The operating system ensures that these devices are able to communicate with the computer, and that they are used in a safe and efficient manner.
- Memory management is responsible for allocating and managing the computer's memory. The operating system ensures that all of the programs that are currently running have enough memory to function properly, and that no one program uses too much memory.
- Network management is responsible for managing the computer's network connections. The operating system allows users to connect to other computers on a network, and it also provides a way for users to share files and resources with each other.
- Security management is responsible for protecting the computer from unauthorized access. The operating system provides a way for users to create passwords and other security measures, and it also helps to prevent viruses and other malware from infecting the computer.

These are just some of the basic components of an operating system. There are many other components that are involved in making an operating system work, but these are some of the most important ones.

Here are some additional important activities that are performed by an operating system:

Job accounting keeps track of the time and resources used by various jobs and users.

- Control over system performance records delays between request for a service and response from the system.
- Error detecting detects and reports errors in hardware or software.
- System performance controlling controls the overall performance of the system, such as the amount of CPU time and memory that is used.

Operating systems are complex software systems that perform a wide variety of tasks. The basic components that I have listed above are essential for the smooth operation of any computer system.

❖ Fundamental system programming and its capability

- Some of the fundamental capabilities of system programming:
 - Low-level control of hardware. System programmers have direct access to the hardware, which allows them to control it in a very precise way. This can be used to improve performance, optimize resource usage, or implement new features.
 - Efficient resource management. System programmers are responsible for managing the computer's resources, such as memory, CPU time, and I/O devices. They must ensure that these resources are used efficiently, so that the computer can run as smoothly as possible.

Department of software engineering

- Security. System programmers are responsible for the security of the computer system. They must implement security measures to protect the system from unauthorized access, viruses, and other malware.
- Inter-process communication. System programmers must be able to communicate between different processes running on the same computer. This allows them to share data and resources, and to coordinate their activities.
- Fault tolerance. System programmers must design systems that are able to handle errors and failures. This includes implementing error detection and recovery mechanisms, as well as ensuring that the system can continue to operate even if some of its components fail.

System programming is a complex and challenging field, but it is also very rewarding. System programmers have the opportunity to work on some of the most important and challenging problems in computer science. They are also responsible for creating the foundation upon which all other software is built.

Here are some of the languages that are commonly used for system programming:

- C is a low-level language that is very efficient and gives the programmer a lot of control over the hardware.
- C++ is a high-level language that is based on C. It is more powerful than C, but it is also more complex.
- Rust is a newer language that is designed to be safe and secure. It is gaining popularity in the system programming community.

System programming is a vast and complex field, but it is also a very rewarding one. If you are interested in learning more about system programming, there are many resources available online and in libraries.